

4) Real-world systems to store in a DB

1) Banking system

→ stores customer details, accounts, transactions, loans, and credit cards

2) School / College management system

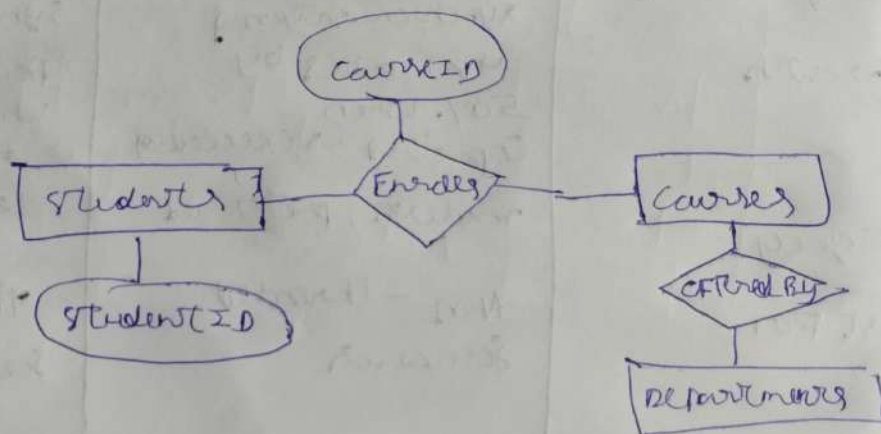
→ stores students, teachers, courses, exams and results

Ex:- Students, courses, Grades

3) Online Food Delivery system:-

→ stores restaurants, menus, orders, payments and delivery details

→ Ex:- Restaurants, menu items, orders, payments



Atomicity

A transaction is all-or-nothing in our case:

→ deduct 1000 from Account A.

→ Add 1000 to Account B

if either step fails, the whole transaction rolls back - so no partial money loss.

2) Consistency:-

A transaction brings the database from one valid state from to another valid state

→ the total money in the system before & after must remain the same

→ if Account A had 5000 and Account B had 2000 after transfer: $A = 4000$
 $B = 3000 \rightarrow \text{total} = 7000$ (unchanged)

3) Isolation

concurrent transactions should not interfere with each other

→ if two people are transferring money at the same time, each transaction should behave as if it is the only one happening

→

4) First Normal Form (1NF):-

→ NO repeating groups / multivalued attributes

→ Each column should hold atomic values only

→ Second Normal Form (2NF):-

must be in 1NF

→ Remove partial dependencies (non-key attributes depending only on part of a composite key).

Third Normal Form (3NF):-

→ must be in 2NF dependencies

→ remove transitive